

API design

REST and gRPC: designing contracts that services — and the teams behind them — can live with.

Contents

01 The contract

02 REST

03 gRPC

04 Choosing a protocol

01

— The contract

APIs as contracts

- An API is a promise: shape, meaning and behavior — not just URLs
- Consumers couple to everything you expose, including your mistakes
- Contract-first: agree on the interface before writing the handlers
- Errors, timeouts and limits are part of the contract — not just the happy path

Qualities of a good API

Consistent

Same naming, casing, pagination and error shape everywhere. Learn one endpoint, and you've learned them all.

Evolvable

Designed so the next change is additive. The contract will outlive the implementation — by years.

Explicit about failure

Every error tells the caller what happened, whose fault it was, and what to do next.

02

— REST

Resources and verbs

METHOD	MEANING	IDEMPOTENT	IN THE STORE
GET	Read, never change	Yes	<code>GET /products/sku-1</code>
POST	Create under a collection	No	<code>POST /orders</code>
PUT	Replace at a known URL	Yes	<code>PUT /products/sku-1</code>
PATCH	Partial update	No	<code>PATCH /orders/42</code>
DELETE	Remove	Yes	<code>DELETE /orders/42</code>

Status codes that matter

CODE	MEANING	WHEN
200 · 201	OK · Created	Successful read; successful create, with a Location header
400	Bad request	The body or parameters fail validation — caller's fault
404	Not found	The resource doesn't exist — or the caller may not know it does
409	Conflict	Valid request, wrong state — paying an already-cancelled order
500 · 503	Server error	A bug vs. overload — only 503 invites the caller to retry

A resource in FastAPI

```
class Product(BaseModel):
    sku: str
    name: str
    price_cents: int

@app.get("/products/{sku}", response_model=Product)
def get_product(sku: str) → Product:
    product = repo.find(sku)
    if product is None:
        raise HTTPException(404, detail="unknown sku")
    return product
```

The model is the contract: validation, serialization and the OpenAPI docs all derive from it.

The same resource in Spring

```
@RestController
@RequestMapping("/products")
class ProductController {

    @GetMapping("/{sku}")
    Product get(@PathVariable String sku) {
        return repo.find(sku).orElseThrow(
            () → new ResponseStatusException(NOT_FOUND));
    }

    @PostMapping @ResponseStatus(CREATED)
    Product create(@RequestBody @Valid Product product) {
        return repo.save(product);
    }
}
```

Versioning and evolution

Additive by default

New optional fields and new endpoints break nobody. Never rename, retype or repurpose a field that has shipped.

Tolerant readers

Clients ignore fields they don't recognize; servers accept what they can. Unknown $\neq$ invalid.

Version when you must

Major version in the path — [/v2/orders](#) — run old and new side by side, and sunset on a date, not by surprise.

Pagination, filtering, errors

Paginate everything

Collections always page: `GET`

`/orders?limit=20&cursor=...` Cursors survive inserts; offsets drift.

Filter in the query

Filters are parameters, not verbs in the path: `/orders?status=paid`, never `/getPaidOrders`.

One error shape

A machine-readable code, a human message, a correlation id — the same envelope on every endpoint (RFC 9457).

03

— gRPC

Protocol buffers

PAYMENTS.PROTO — THE SERVICE

```
syntax = "proto3";

package payments.v1;

service Payments {
    rpc Charge(ChargeRequest)
        returns (ChargeReply);

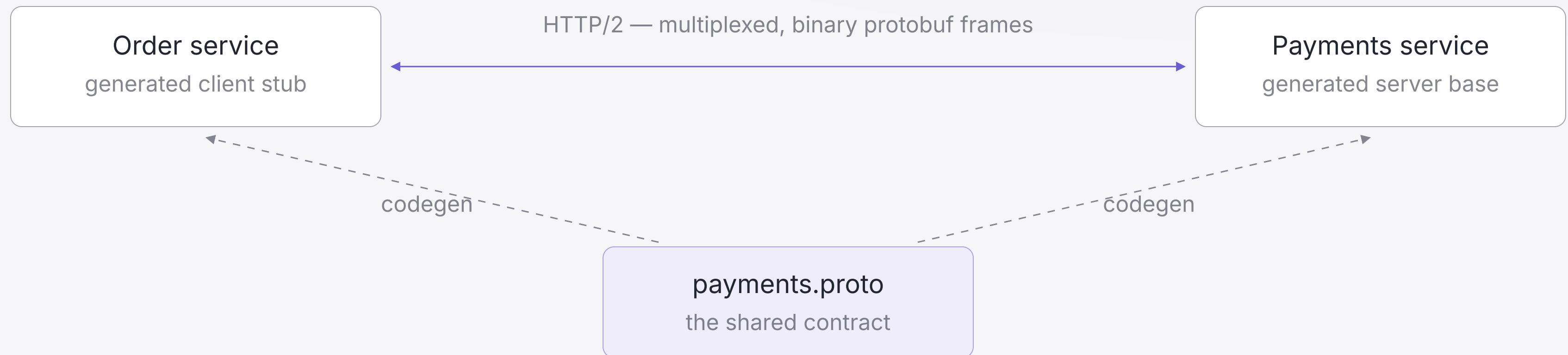
    rpc WatchStatus(WatchRequest)
        returns (stream StatusEvent);
}
```

PAYMENTS.PROTO — THE MESSAGES

```
message ChargeRequest {
    string order_id      = 1;
    int64  amount_cents = 2;
    string currency      = 3;
}

message ChargeReply {
    string charge_id = 1;
    ChargeStatus status = 2; // APPROVED · DECLINED
}
```

How gRPC works



Both sides are generated from one file — the contract cannot drift from the code.

Four call types

Unary

One request, one reply — the RPC twin of a REST call.

`Charge()` is unary.

Server streaming

One request, many replies: progress updates, watch feeds, large result sets. `WatchStatus()`.

Client streaming

Many requests, one reply: uploads, metric batches, log shipping.

Bidirectional

Both sides stream over one connection: chat, live sync, signaling.

← one reply · one request above

a stream of replies · a stream of requests below →

A gRPC service in Java

```
class PaymentsService extends PaymentsGrpc.PaymentsImplBase {

    @Override
    public void charge(ChargeRequest request,
                      StreamObserver<ChargeReply> reply) {
        var charge = processor.charge(
            request.getOrderId(), request.getAmountCents());
        reply.onNext(ChargeReply.newBuilder()
            .setChargeId(charge.id())
            .setStatus(ChargeStatus.APPROVED)
            .build());
        reply.onCompleted();
    }
}
```

04

— Choosing a protocol

REST vs gRPC

DIMENSION	REST	GRPC
Contract	OpenAPI — often written after	Proto file — always written first
Transport	HTTP, JSON text	HTTP/2, binary protobuf
Streaming	Bolted on (SSE, WebSocket)	Four call types built in
Browsers	Universal	Needs a proxy (gRPC-Web)
Best fit	Public and browser-facing APIs	Internal service-to-service calls

A decision guide

- Browser or third-party consumers → REST, documented with OpenAPI
- Internal calls on a hot path → gRPC, with deadlines on every call
- Streaming in either direction → gRPC
- Most estates do both: REST at the edge, gRPC inside — our store included

Exercise — the orders API

- In pairs, 25 minutes — design the store's orders API on paper
- REST: create, fetch, list with pagination, cancel — verbs, URLs, status codes
- Define the error envelope: what does a failed cancel look like?
- gRPC: sketch `payments.proto` — one unary call, one streaming call
- We compare designs against the real system in session 3

Next: microservices

Session 3 — we assemble the store: three services, two protocols, one compose file.